

Fundamentos de Spring Boot

1. Fundamentos de Spring Boot

De la consola al servidor

Programa de consola

arranca



hace su
trabajo



termina

Ciclo de vida corto: tú controlas el ritmo.

Servidor (backend)

arranca



escucha en
un puerto



se queda
esperando

Su «usuario» es otro programa (navegador, app, otro servidor) que le habla por la red.

Dirección IP

Identifica la máquina dentro de la red: qué ordenador es.

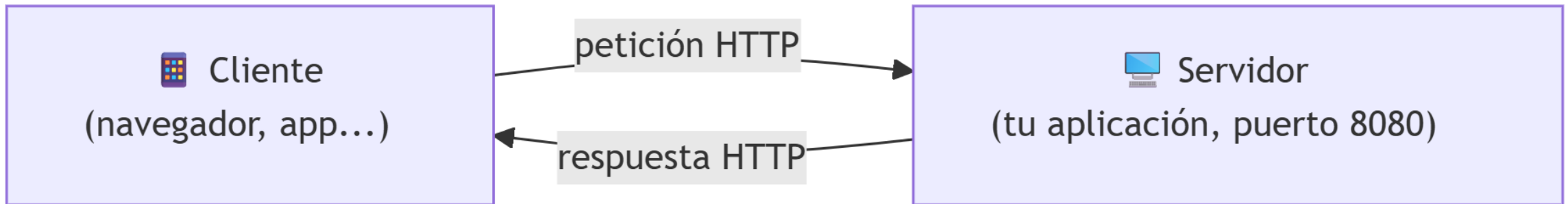
Puerto

Identifica, dentro de esa máquina, a qué programa va dirigido el mensaje.

Como en un bloque de pisos: la IP es la dirección del edificio y el puerto, el número de piso. Un servidor «escuchando en el puerto 8080» espera mensajes dirigidos exactamente a ese piso.

Cliente y servidor hablan HTTP

Cliente y servidor necesitan un formato de mensaje acordado de antemano: un protocolo. El que vas a usar todo el curso es HTTP, el mismo que usa tu navegador al visitar una página.



El servidor no decide cuándo actuar: se queda a la espera y reacciona cada vez que llega una petición. Tu aplicación será justo eso, un programa Java sin Scanner que responde peticiones HTTP hasta que alguien lo detiene.

Qué es un framework

Sin framework

El flujo lo llevas tú: tu `main()` llama a tus clases y estas llaman a las librerías cuando las necesitan.



Con framework

Inversión de control: la relación se invierte y el framework llama a tu código cuando él decide.

Como cocinar en casa o en un restaurante: en casa decides el orden; en el restaurante el camarero avisa «mesa 4, primer plato» y tú solo escribes la receta.

Spring es el framework de Java más usado. Spring Boot le añade:

Servidor web integrado

No instalas nada aparte.

Configuración automática

Se configura según lo que hay en el classpath.

Starters

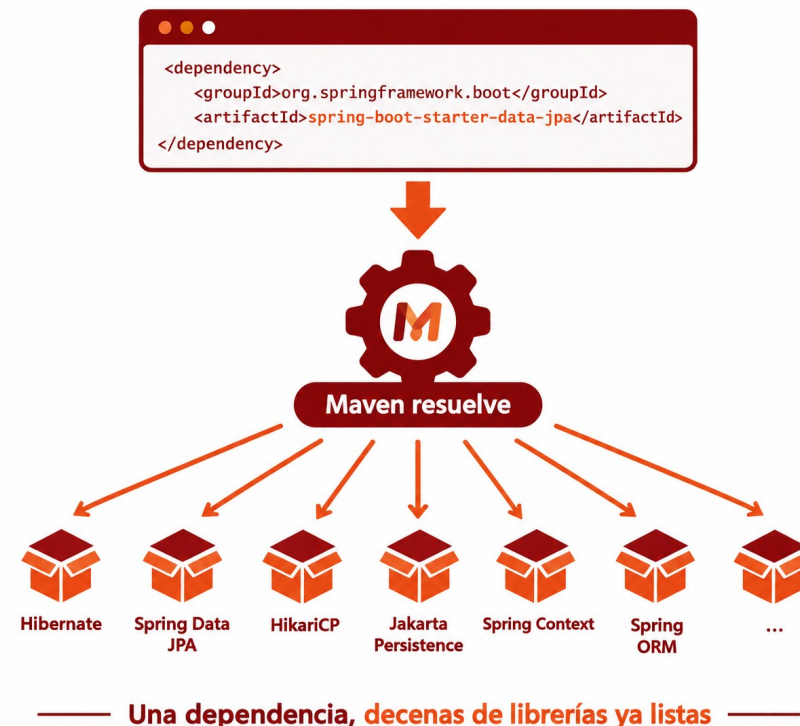
Paquetes que traen de golpe todas las dependencias de una funcionalidad.

Maven y el pom.xml

Maven descarga cada .jar con la versión correcta y resuelve también sus dependencias. Tú solo declaras qué necesitas:

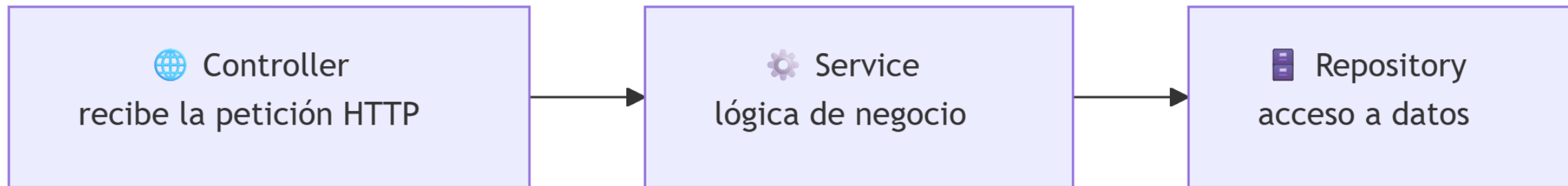
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

En Spring Boot 4, spring-boot-starter-web se separó en starter-webmvc (el que usarás) y starter-webflux (reactivo, fuera del curso). Si buscas starter-web a secas, no lo encontrarás.



groupId: quién publica la librería. artifactId: cuál de sus paquetes quieres.

La arquitectura en capas



Controller

LibroController: recibe la petición HTTP y la traduce a una llamada Java. Solo enruta.

Service

LibroService: la lógica. Qué comprobar, qué calcular y en qué orden.

Repository

LibroRepository: es quien habla con la base de datos.

Cada capa se entiende, se prueba y se cambia sin tocar las demás: si cambia la base de datos, solo el repository. Y cada capa habla solo con la siguiente: el controller nunca accede directamente a la base de datos.

Inyección de dependencias

Sin DI: el service se construye su dependencia

```
public class LibroService {  
    private LibroRepository  
        repositorio =  
            new LibroRepository();  
}
```

El problema

Queda unido a esa implementación concreta. En un test no puedes sustituirla por un repositorio simulado.

Con DI: lo declara por constructor y se lo entregan

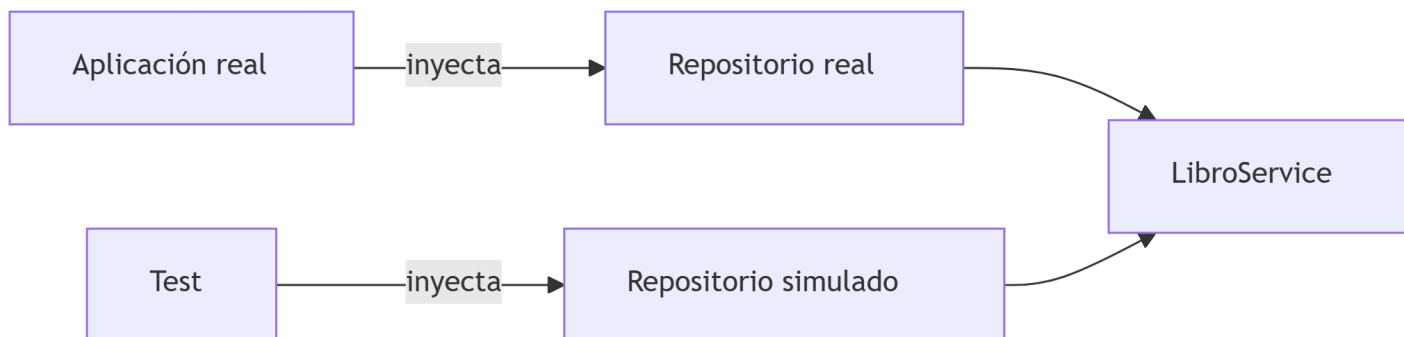
```
public class LibroService {  
  
    private final LibroRepository repositorio;  
  
    public LibroService(  
        LibroRepository repositorio) {  
        this.repositorio = repositorio;  
    }  
}
```

«Para funcionar necesito un LibroRepository»

Inyección de dependencias: una clase recibe desde fuera los objetos que necesita en vez de construirlos ella misma. Es una aplicación concreta de la inversión de control.

Quién entrega las dependencias: Spring

La dependencia se cambia sin tocar el service:



Los beans

Al arrancar, Spring crea y guarda en su contexto una instancia de cada clase gestionada. No vas tú al almacén: te traen los ingredientes a la mesa.

Sin inyección de dependencias:



LibroService no va a buscar su LibroRepository: se lo entregan.

Con inyección de dependencias:



Qué clases gestiona Spring

@Service

Un bean con lógica de negocio: capa intermedia entre controlador y datos.

@Repository

Un bean que accede a datos, normalmente una interfaz que Spring Data implementa por ti.

@RestController

Un bean que expone endpoints HTTP y devuelve las respuestas en el cuerpo (JSON).

@Configuration

Una clase que define a mano uno o varios beans, para los casos que no encajan en las otras.

Las cuatro le dicen a Spring: «esto es un bean, gestiona su ciclo de vida y pásame lo que necesite por inyección».

Clases marcadas con las anotaciones

Spring las crea
→

Contexto de la aplicación
(un bean por clase)

inyecta →

Constructor de cada clase

El arranque y la cadena completa

Toda aplicación tiene una clase con un main()

```
@SpringBootApplication
public class LibreriaApplication {
    static void main(String[] args) {
        SpringApplication.run(
            LibreriaApplication.class, args);
    }
}
```

@SpringBootApplication

Escanea este paquete y los que cuelgan de él buscando beans, y se autoconfigura según el classpath. run(...) arranca el servidor embebido y deja la aplicación escuchando.

La cadena, ya en el controller: recibe el service por constructor

```
@RestController
public class LibroController {
    private final LibroService libroService;
    public LibroController(LibroService libroService) {
        this.libroService = libroService;
    }
    @GetMapping("/{id}")
    public ResponseEntity<LibroDTO> getById(
        @PathVariable Long id) {
        return ResponseEntity.ok(libroService.findById(id));
    }
}
```

Solo interesa la forma

controller → service → repository, cada uno con su responsabilidad y sin saltarse capas. @GetMapping, @PathVariable y ResponseEntity son de HTTP/REST: los verás en PSP.

Lombok y configuración

@RequiredArgsConstructor genera el constructor

```
@Service
@RequiredArgsConstructor
public class LibroService {
    private final LibroRepository libroRepository;
    private final EditorialRepository
        editorialRepository;
    // Lombok genera aquí, al compilar,
    // un constructor con ambos repositorios.
}
```

Un campo final más, sin tocar nada

Sin Lombok escribirías el constructor a mano cada vez. Spring inyecta igual, por constructor.

application.yml

Configuración común: credenciales, puertos, propiedades propias.

application-dev.yml

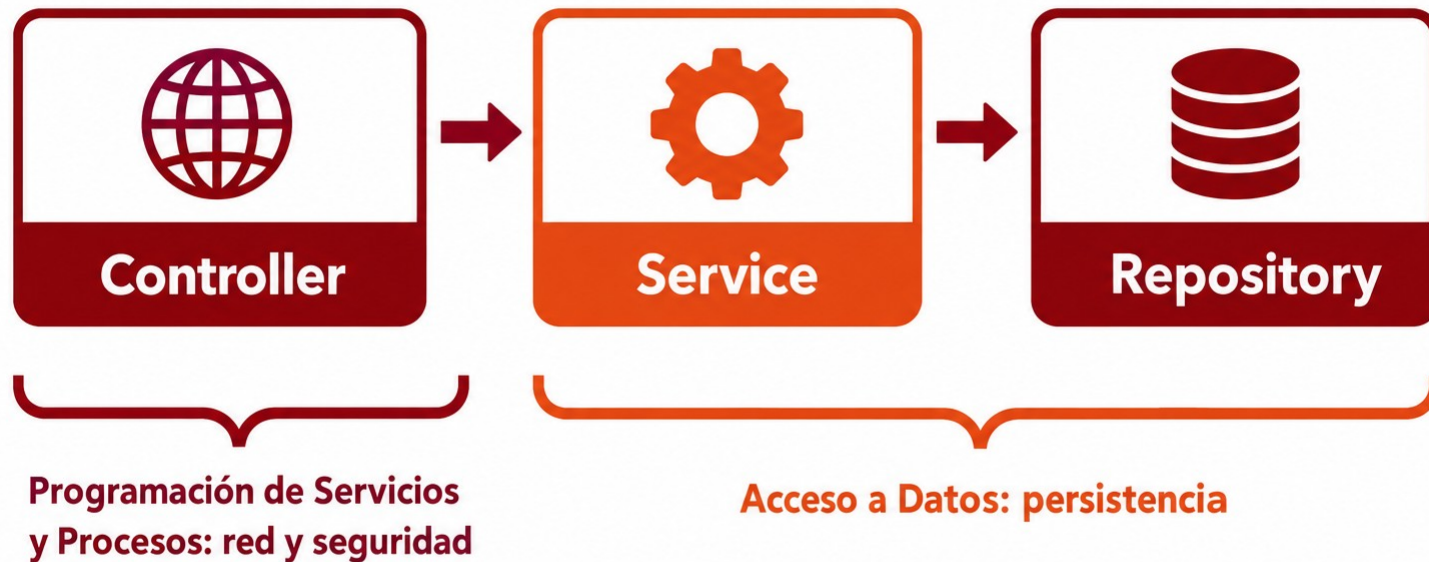
La del entorno de desarrollo, por ejemplo la URL de PostgreSQL de la Actividad 1.1.

El «-dev» es un perfil

Solo se activa si lo pides al arrancar. Mismo código, otra configuración.

.yml o .yaml: Spring Boot reconoce las dos extensiones. En este curso se usa siempre .yml, por uniformidad.

Dónde estás y hacia dónde vas



Acceso a Datos

Persistencia: del service hacia abajo (JDBC, JPA, JSONB, MongoDB).

PSP

Servicios en red y seguridad: del controller hacia fuera (REST, JWT, hilos, sockets).

En las actividades de ambos módulos construirás, pieza a pieza, tu propia aplicación con esta estructura.

Ideas clave

1

Un servidor arranca y se queda escuchando en un puerto. La IP identifica la máquina, el puerto el programa y HTTP es el protocolo.

2

Un framework invierte el control. Spring Boot añade configuración automática, servidor embebido y starters.

3

Maven gestiona las dependencias vía pom.xml: starter-webmvc y starter-data-jpa son las claves del curso.

4

Capas: controller (HTTP) → service (lógica) → repository (datos). Cada una habla solo con la siguiente.

5

DI: la clase declara lo que necesita por constructor y Spring se lo entrega. @Service, @Repository, @RestController y @Configuration marcan lo que gestiona.

6

@RequiredArgsConstructor genera el constructor. application.yml y application-dev.yml centralizan la configuración: un perfil la varía sin tocar código.